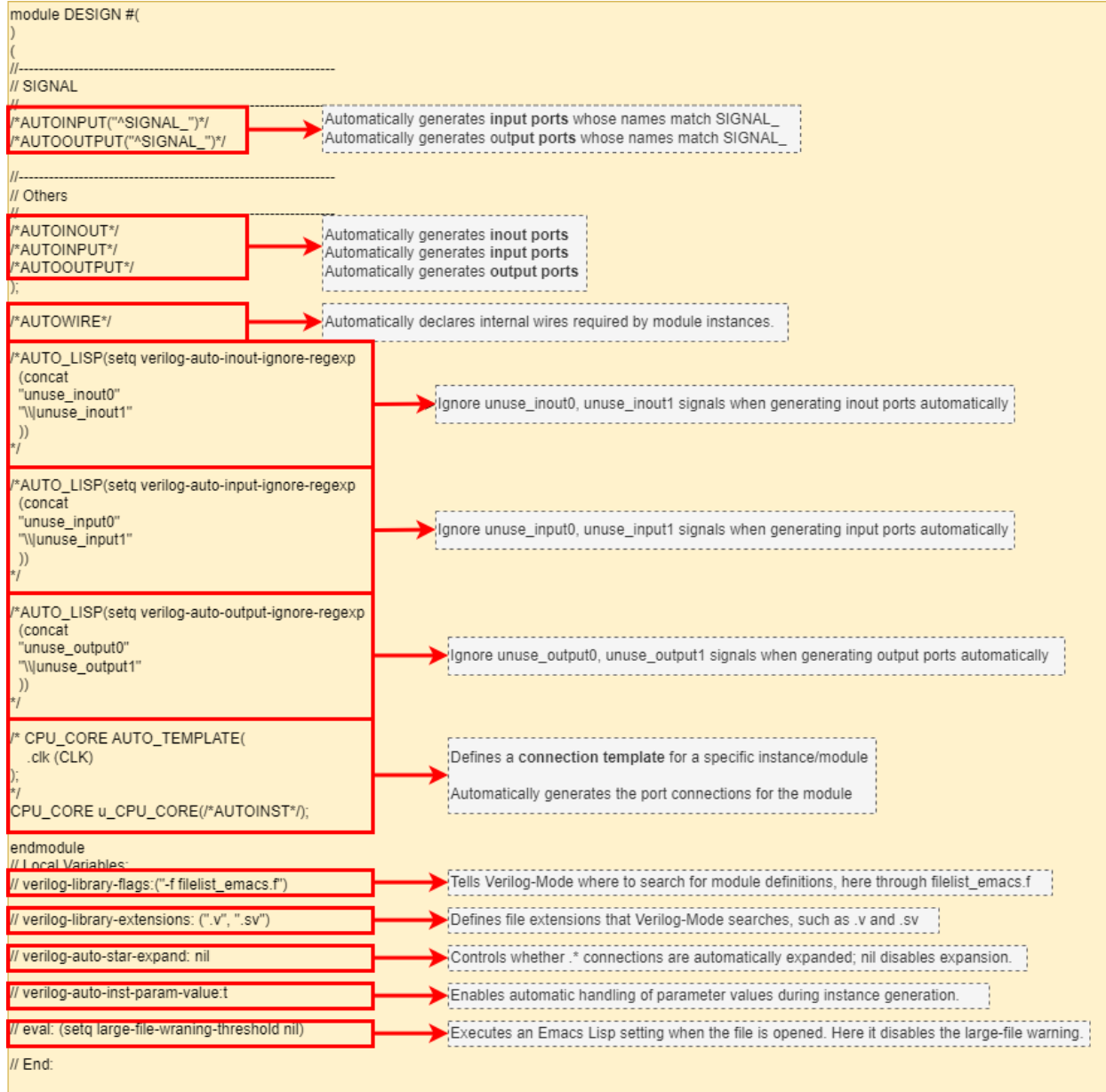


Quick integration guide with EMACS (Verilog Mode)

Introduction: In integration tasks, Verilog-Mode in Emacs helps automate module connections and signal declarations, reducing repetitive manual work. It improves integration speed, consistency, and reliability, especially for large hierarchical RTL designs.

Example of Verilog-Mode Integration Template:



Most common use functions

Function	Usage	Description
<code>\(.*)</code>	As is: <code>.INT_\(.*) (INTERRUPT_1[]),</code> To be: <code>.INT_WDT_TIMEOUT (INTERRUPT_WDT_TIMEOUT),</code>	Find any of words or numbers
<code>@</code>	As is: <code>.REG@_\(.*) (\2[]),</code> To be: <code>.REG1_CNT (CNT),</code>	Find numbers
<code>\1, \2, \3, ...</code>	As is: <code>.REG@_\(.*) (\2[]),</code> To be: <code>.REG1_CNT (CNT),</code>	First/Second/Third... words or numbers found
<code>\([A-Z]+\)</code>	As is: <code>.CLK_\([A-Z]+\)CLK (\1CLK),</code> To be: <code>.CLK_CPUCLK (CPUCLK),</code>	Find words
downcase	As is: <code>.INT_\(.*) (@)"(downcase (symbol-name '\1))"[]),</code> To be: <code>.INT_WDT_TIMEOUT (wdt_timeout),</code>	Replace upper case letter to lower case letter
upcase	As is: <code>.int_\(.*) (@)"(upcase (symbol-name '\1))"[]),</code> To be: <code>.int_wdt_timeout (WDT_TIMEOUT),</code>	Replace lower case letter to upper case letter
<code>[]</code>	As is: input wire [3:0] WDT_TIMEOUT, <code>.int_\(.*) (@)"(upcase (symbol-name '\1))"[]),</code> To be: <code>.int_wdt_timeout (WDT_TIMEOUT[3:0]),</code>	Takes the bit range for the bus from the referenced module
if statement	As is: <code>.ERR_\(.*) (@)"(if{equal vl-dir \\"input\\"} \\"0\\" \\"\\")",</code> To be: <code>.ERR_OUTPUT (),</code> <code>.ERR_INPUT ('0),</code>	Has same functionality with usual if statement
vl-name	As is: <code>.INT_\(.*) (@)"(downcase vl-name)"[]),</code> To be: <code>.INT_WDT_TIMEOUT (int_wdt_timeout),</code>	Name portion of the input/output port
vl-width	As is: <code>.INT_\(.*) ({@"vl-width"{1'b0}}),</code>	Width of the input/output port

	To be: <code>.INT_WDT_TIMEOUT ({4{1'b0}}),</code>	
vl-dir	As is: <code>.ERR_\(.*) (@”(if(equal vl-dir \”input\”) \”0\” \”\”))”,</code> To be: <code>.ERR_OUTPUT (),</code> <code>.ERR_INPUT (’0),</code>	Direction of the input/output/inout port
symbol-name	As is: <code>.INT_\(.*) (@”(downcase (symbol-name ‘\1))”[]),</code> To be: <code>.INT_WDT_TIMEOUT (wdt_timeout),</code>	Create symbol’s name
vl-cell-name	As is: <code>.INT (int_@”(substring vl-cell-name 11 -2))”,</code> <code>WDT u_instance_wdt_0 (),</code> To be: <code>.INT (int_wdt),</code>	Get instance name of the cell (vl-cell-name is u_instance_wdt_0)
substring	As is: <code>.INT (int_@”(substring vl-cell-name 11 -2))”,</code> <code>WDT u_instance_wdt_0 (),</code> To be: <code>.INT (int_wdt),</code>	Split instance name string into pieces Substring vl-cell-name 11 -2: Delete 11 characters from left, delete 2 characters from right, keep the string between.
bit width inside [] calculate	Hardcode style: <code>.int_0 (int[3:0]),</code> <code>.int_1 (int[7:4]),</code> <code>.int_2 (int[11:8]),</code> Using emacs regex: <code>.int_@ (int[@”(- (* (+ \1 1) 4) 1)”:@” (* \1 4)”]),</code>	Normal writing: 1. A+B 2. (A+B)*2 3. (A-B)%4 Emacs regex: 1. + A B 2. * (+ A B) 2 3. % (- A B) 4 @ template, assume A is \1 and B is \2 1. + \1 \2 2. * (+ \1 \2) 2 3. % (- \1 \2) 4
	As is: CPU_IP AUTO_TEMPLATE “_\([0-9]\)\\$” (<code>.SFR_\(.*) (SFR_@_\1[]),</code>); CPU_IP u_cpu_0(/*AUTOINST*/); CPU_IP u_cpu_1(/*AUTOINST*/); To be: CPU_IP u_cpu_0 (	Get pattern from instance name

	<pre>.SFR_REG (SFR_0_REG),); CPU_IP u_cpu_1 (.SFR_REG (SFR_1_REG),);</pre>	
--	---	--

Reference:

1/ [Veripool – Verilog-Mode Documentation](#)

2/ [GitHub – veripool/verilog-mode](#)